ResearchRAG AI

Intelligent Academic Research Assistant

Final Technical Report

1. Executive Summary

ResearchRAG AI is an end-to-end academic assistant built on arXiv metadata. The system combines semantic retrieval, paper classification, and citation-grounded question answering within a Streamlit application.

The project addresses a key research workflow challenge: users often need to search for papers, classify them, and generate evidence-backed insights. Traditional keyword search is insufficient due to variations in terminology across research papers.

The system integrates four major capabilities:

- Semantic search using sentence-transformer embeddings and FAISS
- Paper classification using locally trained models
- Fallback mode using local retrieval when API access is unavailable
- Citation-grounded question answering using Google Gemini over retrieved arXiv context

2. Problem Statement

Academic research workflows are fragmented across multiple tools. Users must rely on separate systems for:

- Search
- Categorization
- Summarization

This leads to inefficiency and weak integration between retrieval and answer generation.

ResearchRAG AI solves this by providing a unified, retrieval-first system.

3. Objectives

The system was designed to:

- Classify research papers using title and abstract
- Retrieve semantically similar papers
- Generate evidence-based answers
- Provide fallback functionality when external API access is unavailable

- Deliver a polished user interface via Streamlit

4. Dataset and Data Preparation

4.1 Dataset Source

- Dataset: arXiv Metadata Snapshot
- Platform: Kaggle
- URL: https://www.kaggle.com/datasets/Cornell-University/arxiv

4.2 Dataset Snapshot

- Retrieval dataset size: **5,000 records**
- Unique categories: **122**
- Classification dataset: **2,740 samples**
- Training set: **2,192**
- Test set: **548**
- Target categories: **Top 10 classes**

4.3 Exploratory Data Analysis

Key findings:

- Dataset is **highly imbalanced**
- Dominant category: `astro-ph`
- Average title length: **70.43 characters**
- Average abstract length: **802.50 characters**

The top-10 category approach reduced sparsity and improved classification reliability.

4.4 Data Pipeline

1. Load arXiv metadata
2. Combine title and abstract
3. Generate text chunks
4. Create embeddings
5. Build FAISS index
6. Train classification models
7. Save deployment artifacts

5. System Architecture

The system follows a **Retrieval-Augmented Generation (RAG)** architecture:

- Data preprocessing from arXiv
- Embeddings using `all-MiniLM-L6-v2`
- FAISS vector index for search
- Local classification models
- Gemini for retrieval-grounded answer generation
- Streamlit frontend

Runtime Flow

1. User inputs query
2. System retrieves relevant documents
3. Either:
    - Generates answer using Gemini
    - OR produces fallback answer using local data

6. Modeling Approach

6.1 Classification Models

Logistic Regression

- TF-IDF (unigrams + bigrams)
- Max features: 6,000
- Role: Baseline model

MLP Classifier

- TF-IDF (unigrams + bigrams)
- Max features: 2,000
- Hidden layer: 64 neurons
- Role: Non-linear classification

6.2 Retrieval Model

- Embedding model: `all-MiniLM-L6-v2`
- Vector database: FAISS
- Stored metadata for efficient retrieval

6.3 Generation Model

Google Gemini is used with constraints:

- Uses only retrieved context
- Avoids hallucination
- Provides inline citations
- Indicates insufficient context clearly

7. Evaluation Results

7.1 Classification Metrics

Logistic Regression

- Accuracy: **0.7938**
- Macro Precision: **0.8123**
- Macro Recall: **0.6785**
- Macro F1-score: **0.7214**

MLP Classifier

- Accuracy: **0.8303**
- Macro Precision: **0.8075**
- Macro Recall: **0.7480**
- Macro F1-score: **0.7690**

7.2 Best Model

The **MLP classifier** performed best based on Macro F1-score.

7.3 Interpretation

- Both models performed well
- MLP improved minority class recall
- Macro metrics were crucial due to imbalance

8. Deployment

8.1 Application Interface

The Streamlit app includes:

- Ask
- Discover

- Classify
- Insights

8.2 Deployment Artifacts

- FAISS index
- Embedding metadata
- Trained classification models
- Label encoder
- EDA visualizations

8.3 Configuration

- Gemini API via environment variables
- Fallback mode if API unavailable

8.4 Resilience Features

- Cached retrieval components
- Local fallback answers
- Precomputed embeddings
- Offline-compatible design

9. Product Impact

9.1 Faster Literature Discovery

Semantic search improves relevance beyond keyword matching

9.2 Research Assistance

Answers are grounded in retrieved evidence

9.3 Classification Support

Quick categorization of papers

9.4 Deployment-Ready Tool

Fully usable system with UI and backend integration

10. Limitations

- Limited dataset subset (5,000 records)
- Only top-10 categories classified

- Depends on retrieval quality
- Fallback responses less fluent
- Raw dataset not included in deployment

11. Future Improvements

- Expand category coverage
- Improve retrieval evaluation metrics
- Add structured citations export
- Enable multi-paper comparison
- Add conversational memory
- Integrate additional academic sources

12. Conclusion

ResearchRAG AI successfully integrates:

- Machine learning
- Semantic retrieval
- Large language models
- Deployment engineering
- Into a single academic assistant

The system meets all core objectives and provides a strong foundation for future AI-powered research tools.